REAL PROCESSES. VISIBLE CONCEPTS.

ForkLab

Parent-Child Process & IPC Visualizer

Making process creation, program execution and communication observable.

TEAM 12 | GUIDE: HARIKA | KLH BACHUPALLY, CSE

G.ROHITH (2520030220) | SARVESH (2520030049)
GAUTHAM (2520030316)

A command runs. But what happens underneath?

The learning gap

Output alone does not explain:

Which process created the child?
How did the child load a program?
How does the parent collect its status?

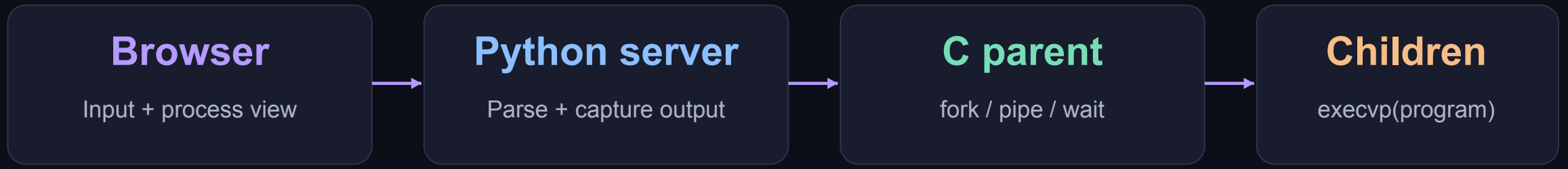
Our objective

Connect OS theory to real execution.

Show parent and child PIDs.
Explain fork, exec and wait.
Visualize communication through pipes.

The command input triggers the experiment. The process lifecycle is the lesson.

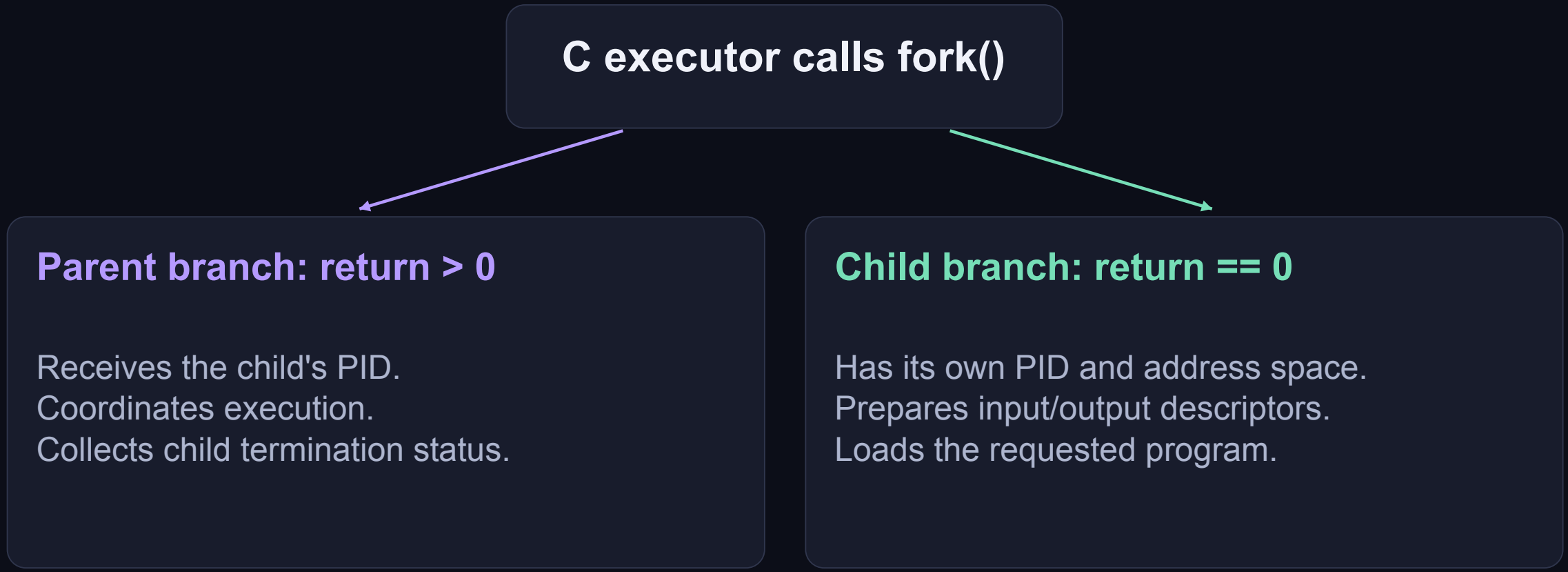
Web interface outside. C process engine inside.



Return path: stdout + stderr + execution metadata

Python captures the results. The browser displays output, process IDs, system-call traces, duration and exit status.

One parent. A new child. Two execution paths.



Failure path: fork() returns -1; report the error rather than pretending success.

Create. Replace. Collect.

fork()

Creates a new child.

New PID.
Separate process.
Parent and child continue.

execvp()

Loads a new program.

Same child PID.
New program image.
Success does not return.

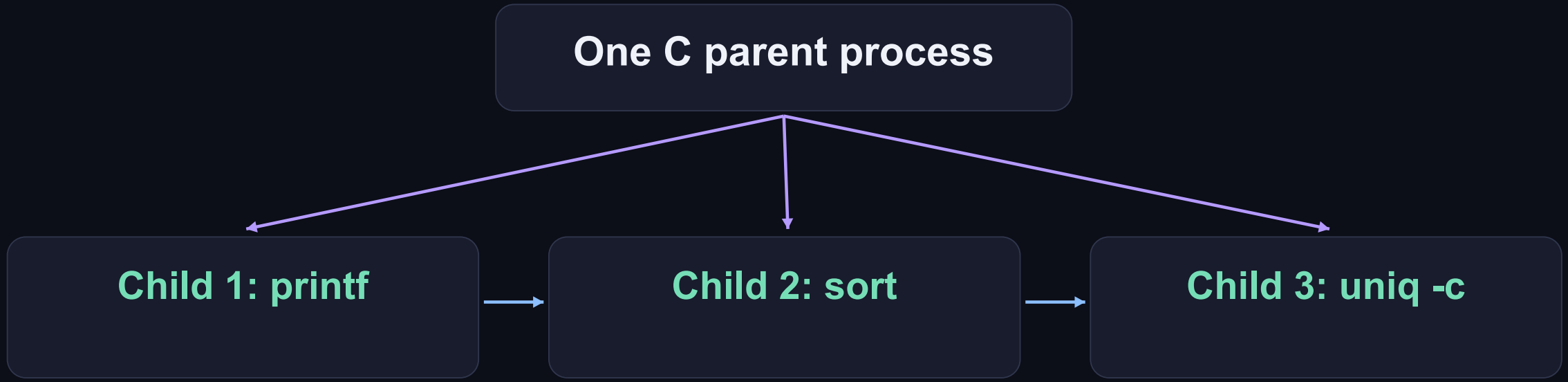
waitpid()

Collects child status.

Parent waits for its child.
Checks how it terminated.
Reaps the child entry.

execvp() does not create another process. It replaces the existing child.

Three sibling children. Two real pipes.

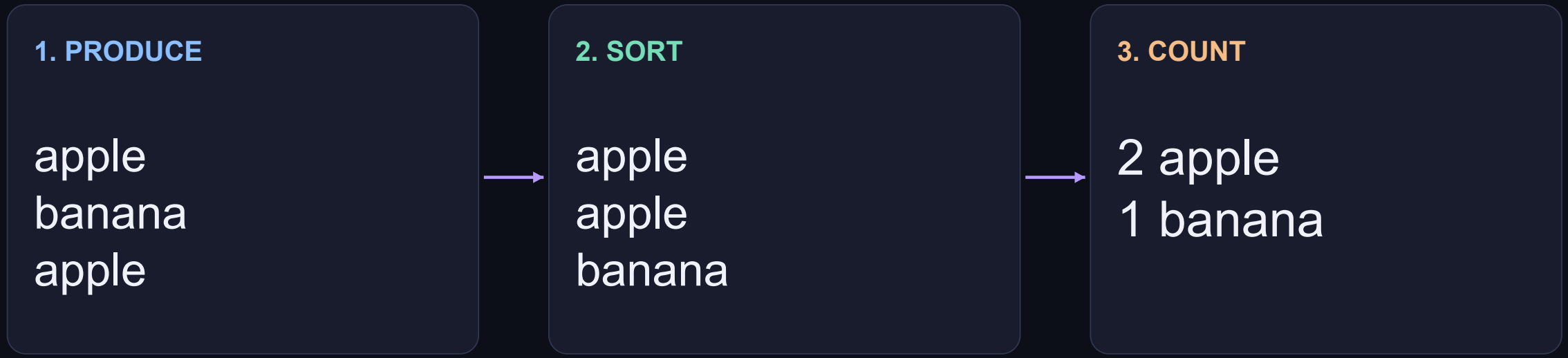


pipe() creates channels. dup2() connects stdin and stdout.

Close unused pipe ends so readers can detect EOF.
The parent uses waitpid() to collect every child.

From three lines to two frequency counts.

```
printf "apple\nbanana\napple\n" | sort | uniq -c
```



Why sort first? uniq groups adjacent duplicates, not all matches anywhere.

A small C engine with observable behavior.

```
pipe(channel);
pid_t pid = fork();
if (pid == 0) {
dup2(channel[1], STDOUT_FILENO);
execvp(args[0], args);
_exit(127); // reached on exec failure
}
waitpid(pid, &status, 0);
```

Illustrative excerpt only; full code includes checks, closure and multi-child handling.

Implementation responsibilities

C: create children, connect descriptors and collect statuses.

Python: parse input, manage jobs and capture results.

JavaScript: display observed output and process traces.

Show evidence, not just terminal output.

01 Create one child

printf "Hello Faculty\n"

Point out the parent PID, child PID and exit status.

02 Connect three children

printf "apple\nbanana\napple\n" | sort | uniq -c

Show 3 child PIDs, 2 pipes and the expanded trace.

03 Observe an execution failure

forklab_command_not_found

Explain execvp() failure and child exit status 127.

Demo URL: http://127.0.0.1:8001/ | Restart: make web; use the printed URL.

A process lab, with room to explore.

Process monitor + history

Real system snapshots; searchable local execution records.

Stop + bounded execution

Process-group cancellation; 12s timeout and 256 KiB output limit in v2.

CPU scheduling simulation

FCFS, non-preemptive SJF and Round Robin with Gantt charts.

Guided experiments

Runnable lessons covering processes, pipes, files, failures and signals.

Real execution and educational simulation are clearly separate.

Make the operating system understandable.

What we demonstrate

Real parent-child process creation.

Program replacement with execvp().

Pipe-based inter-process communication.

Termination-status collection with waitpid().

What we do not claim

Not a full replacement for Terminal.

Not a sandbox: commands use real user permissions.

Scheduling charts do not control the kernel scheduler.

No automatic cloud upload of laptop data.

Our outcome: OS concepts become visible, testable and easier to explain.

The five answers to remember.

fork vs. exec?

fork creates a child. exec replaces its program; the PID stays.

Why waitpid()?

Collect exit status and reap the child process entry.

Are pipeline stages a process chain?

No. They are siblings; pipes connect their data streams.

Why sort before uniq?

uniq counts consecutive duplicates, so equal lines must be adjacent.

How is this different from Terminal?

The focus is explaining real process creation and IPC.

References: POSIX/Linux man-pages (fork, execvp, pipe, dup2, waitpid); Python docs; OSSP material.